In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
!pip install seaborn
```

```
Collecting seaborn
  Using cached seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in .\.venv\Lib\site-packages (fr
om seaborn) (2.5.4)
Requirement already satisfied: pandas>=1.2 in .\.venv\Lib\site-packages (from seabor
n) (3.1.0)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in .\.venv\Lib\site-packages
(from seaborn) (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in .\.venv\Lib\site-packages (from m
atplotlib!=3.6.1,>=3.4->seaborn) (1.4.0)
Requirement already satisfied: cycler>=0.10 in .\.venv\Lib\site-packages (from matpl
otlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in .\.venv\Lib\site-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\.venv\Lib\site-packages (from
matplotlib!=3.6.1,>=3.4->seaborn) (1.5.1)
Requirement already satisfied: packaging>=20.0 in .\.venv\Lib\site-packages (from ma
tplotlib!=3.6.1,>=3.4->seaborn) (26.3)
Requirement already satisfied: pillow>=9 in .\.venv\Lib\site-packages (from matplotl
ib!=3.6.1,>=3.4->seaborn) (13.0.0)
Requirement already satisfied: pyparsing>=3 in .\.venv\Lib\site-packages (from matpl
otlib!=3.6.1,>=3.4->seaborn) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in .\.venv\Lib\site-packages (fr
om matplotlib!=3.6.1,>=3.4->seaborn) (3.0.0)
Requirement already satisfied: tzdata in .\.venv\Lib\site-packages (from pandas>=1.2
->seaborn) (2026.5)
Requirement already satisfied: six>=1.6 in .\.venv\Lib\site-packages (from python-da
teutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Using cached seaborn-0.13.2-py3-none-any.whl (294 kB)
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
```

In [5]:
```
data = {
    "Order_ID": [
        101, 102, 103, 104, 105, 106,
        107, 108, 109, 110, 111, 102
    ],
    "Customer": [
        "Arjun", "Priya", "Rahul", "Sneha",
        "Kiran", "Divya", "Aman", "Neha",
        "Ravi", "Meena", "Vikram", "Priya"
    ],
    "City": [
        "Hyderabad", "Chennai", " hyderabad ",
        "Bangalore", "Chennai", None,
        "Mumbai", "BANGALORE", "Mumbai",
        "Chennai", "Hyderabad", "Chennai"
    ],
```

```
    "Restaurant": [
        "Spice Hub", "Food Court", "Spice Hub",
        "Green Bowl", "Food Court", "Spice Hub",
        "Urban Bites", "Green Bowl", "Urban Bites",
        "Food Court", "Spice Hub", "Food Court"
    ],
    "Order_Date": [
        "2026-10-01", "02/10/2026", "2026-10-03",
        "04/10/2026", "2026-10-05", "31/02/2026",
        "2026-10-07", "not available", "09/10/2026",
        "2026-10-10", None, "02/10/2026"
    ],
    "Quantity": [
        2, "3", 1, "two", 4, 2,
        None, 3, 2, "5", 1, "3"
    ],
    "Price": [
        250, "180.50", 320, 150, None, 220,
        "invalid", 275, 400, 160, 300, "180.50"
    ],
    "Rating": [
        4.5, "4", 5, None, 3.5, "five",
        4, 4.5, None, 3, 4.5, "4"
    ]
}

df = pd.DataFrame(data)

print("Dataset created successfully!")
```

```
Dataset created successfully!
```

In [6]:
```
df.head()
```

Out[6]:

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating
0	101	Arjun	Hyderabad	Spice Hub	2026-10-01	2	250	4.5
1	102	Priya	Chennai	Food Court	02/10/2026	3	180.50	4
2	103	Rahul	hyderabad	Spice Hub	2026-10-03	1	320	5
3	104	Sneha	Bangalore	Green Bowl	04/10/2026	two	150	None
4	105	Kiran	Chennai	Food Court	2026-10-05	4	None	3.5

In [7]:
```
df.tail()
```

Out[7]:

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating
7	108	Neha	BANGALORE	Green Bowl	not available	3	275	4.5
8	109	Ravi	Mumbai	Urban Bites	09/10/2026	2	400	None
9	110	Meena	Chennai	Food Court	2026-10-10	5	160	3
10	111	Vikram	Hyderabad	Spice Hub	NaN	1	300	4.5
11	102	Priya	Chennai	Food Court	02/10/2026	3	180.50	4

In [8]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   Order_ID    12 non-null     int64 
 1   Customer    12 non-null     str   
 2   City        11 non-null     str   
 3   Restaurant  12 non-null     str   
 4   Order_Date  11 non-null     str   
 5   Quantity    11 non-null     object
 6   Price       11 non-null     object
 7   Rating      10 non-null     object
dtypes: int64(1), object(3), str(4)
memory usage: 900.0+ bytes
```

In [9]:
```
df.describe()
```

Out[9]:

	Order_ID
count	12.000000
mean	105.666667
std	3.366502
min	101.000000
25%	102.750000
50%	105.500000
75%	108.250000
max	111.000000

In [10]:
```
df.shape
```

Out[10]: (12, 8)

In [11]:
```
df.columns
```

Out[11]:
```
Index(['Order_ID', 'Customer', 'City', 'Restaurant', 'Order_Date', 'Quantity',
       'Price', 'Rating'],
      dtype='str')
```

In [12]:
```
df.isnull().sum()
```

Out[12]:
```
Order_ID      0
Customer      0
City          1
Restaurant    0
Order_Date    1
Quantity      1
Price         1
Rating        2
dtype: int64
```

In [13]:
```
df.duplicated().sum()
```

Out[13]:
```
np.int64(1)
```

In [14]:
```
df.describe(include="all")
```

Out[14]:

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating
count	12.000000	12	11	12	11	11.0	11	10.0
unique	NaN	11	6	4	10	7.0	10	7.0
top	NaN	Priya	Chennai	Spice Hub	02/10/2026	2.0	180.50	4.5
freq	NaN	2	4	4	2	3.0	2	3.0
mean	105.666667	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	3.366502	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	101.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	102.750000	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	105.500000	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	108.250000	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	111.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN

describe() summarizes numeric columns by default. For a summary of text columns too, try:

In [15]:
```
display(df.dtypes)
```

```
Order_ID       int64
Customer         str
City             str
Restaurant       str
Order_Date       str
Quantity      object
Price         object
Rating        object
dtype: object
```

In [16]:
```
df[df["Price"].isna()]
```

Out[16]:

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating
4	105	Kiran	Chennai	Food Court	2026-10-05	4	None	3.5

In [18]:
```
df[df.isna().any(axis=1)]
```

Out[18]:

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating
3	104	Sneha	Bangalore	Green Bowl	04/10/2026	two	150	None
4	105	Kiran	Chennai	Food Court	2026-10-05	4	None	3.5
5	106	Divya	NaN	Spice Hub	31/02/2026	2	220	five
6	107	Aman	Mumbai	Urban Bites	2026-10-07	None	invalid	4
8	109	Ravi	Mumbai	Urban Bites	09/10/2026	2	400	None
10	111	Vikram	Hyderabad	Spice Hub	NaN	1	300	4.5

In [20]:
```
df["Quantity"].astype(int)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[20], line 1
----> 1 df["Quantity"].astype(int)

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\generic.py:6781, in
NDFrame.astype(self, dtype, copy, errors)
   6777             results = [ser.astype(dtype, errors=errors) for _, ser in self.i
tems()]
   6778
   6779         else:
   6780             # else, only a single dtype is given
-> 6781             new_data = self._mgr.astype(dtype=dtype, errors=errors)
   6782             res = self._constructor_from_mgr(new_data, axes=new_data.axes)
   6783             return res.__finalize__(self, method="astype")
   6784

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\internals\managers.
py:632, in BaseBlockManager.astype(self, dtype, errors)
    631 def astype(self, dtype, errors: str = "raise") -> Self:
--> 632     return self.apply("astype", dtype=dtype, errors=errors)

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\internals\managers.
py:456, in BaseBlockManager.apply(self, f, align_keys, **kwargs)
    454         applied = b.apply(f, **kwargs)
    455     else:
--> 456         applied = getattr(b, f)(**kwargs)
    457     result_blocks = extend_blocks(applied, result_blocks)
    459 out = type(self).from_blocks(result_blocks, [ax.view() for ax in self.axes])

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\internals\blocks.p
y:623, in Block.astype(self, dtype, errors, squeeze)
    620         raise ValueError("Can not squeeze with more than one column.")
    621     values = values[0, :]  # type: ignore[call-overload]
--> 623 new_values = astype_array_safe(values, dtype, errors=errors)
    625 new_values = maybe_coerce_values(new_values)
    627 refs = None

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\dtypes\astype.py:24
5, in astype_array_safe(values, dtype, copy, errors)
    242     dtype = dtype.numpy_dtype
    244 try:
--> 245     new_values = astype_array(values, dtype, copy=copy)
    246 except (ValueError, TypeError):
    247     # e.g. _astype_nansafe can fail on object-dtype of strings
    248     #  trying to convert to float
    249     if errors == "ignore":

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\dtypes\astype.py:19
0, in astype_array(values, dtype, copy)
    187     values = values.astype(dtype, copy=copy)
    189 else:
--> 190     values = _astype_nansafe(values, dtype, copy=copy)
    192 # in pandas we don't store numpy str dtypes, so convert to object
    193 if isinstance(dtype, np.dtype) and issubclass(values.dtype.type, str):
```

```
File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\dtypes\astype.py:139, in _astype_nansafe(arr, dtype, copy, skipna)
    135     raise ValueError(msg)
    137 if copy or object in (arr.dtype, dtype):
    138     # Explicit copy, or required since NumPy can't view from / to object.
--> 139     return arr.astype(dtype, copy=True)
    141 return arr.astype(dtype, copy=copy)

ValueError: invalid literal for int() with base 10: 'two'
```

Expected: This raises a ValueError because the column contains "two" and missing values.

In [21]:
```
df["Price"].astype(float)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[21], line 1
----> 1 df["Price"].astype(float)

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\generic.py:6781, in
NDFrame.astype(self, dtype, copy, errors)
   6777             results = [ser.astype(dtype, errors=errors) for _, ser in self.i
tems()]
   6778
   6779         else:
   6780             # else, only a single dtype is given
-> 6781             new_data = self._mgr.astype(dtype=dtype, errors=errors)
   6782             res = self._constructor_from_mgr(new_data, axes=new_data.axes)
   6783             return res.__finalize__(self, method="astype")
   6784

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\internals\managers.
py:632, in BaseBlockManager.astype(self, dtype, errors)
    631 def astype(self, dtype, errors: str = "raise") -> Self:
--> 632     return self.apply("astype", dtype=dtype, errors=errors)

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\internals\managers.
py:456, in BaseBlockManager.apply(self, f, align_keys, **kwargs)
    454         applied = b.apply(f, **kwargs)
    455     else:
--> 456         applied = getattr(b, f)(**kwargs)
    457     result_blocks = extend_blocks(applied, result_blocks)
    459 out = type(self).from_blocks(result_blocks, [ax.view() for ax in self.axes])

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\internals\blocks.p
y:623, in Block.astype(self, dtype, errors, squeeze)
    620         raise ValueError("Can not squeeze with more than one column.")
    621     values = values[0, :]  # type: ignore[call-overload]
--> 623 new_values = astype_array_safe(values, dtype, errors=errors)
    625 new_values = maybe_coerce_values(new_values)
    627 refs = None

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\dtypes\astype.py:24
5, in astype_array_safe(values, dtype, copy, errors)
    242     dtype = dtype.numpy_dtype
    244 try:
--> 245     new_values = astype_array(values, dtype, copy=copy)
    246 except (ValueError, TypeError):
    247     # e.g. _astype_nansafe can fail on object-dtype of strings
    248     #  trying to convert to float
    249     if errors == "ignore":

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\dtypes\astype.py:19
0, in astype_array(values, dtype, copy)
    187     values = values.astype(dtype, copy=copy)
    189 else:
--> 190     values = _astype_nansafe(values, dtype, copy=copy)
    192 # in pandas we don't store numpy str dtypes, so convert to object
    193 if isinstance(dtype, np.dtype) and issubclass(values.dtype.type, str):
```

```
File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\dtypes\astype.py:139, in _astype_nansafe(arr, dtype, copy, skipna)
    135     raise ValueError(msg)
    137 if copy or object in (arr.dtype, dtype):
    138     # Explicit copy, or required since NumPy can't view from / to object.
--> 139     return arr.astype(dtype, copy=True)
    141 return arr.astype(dtype, copy=copy)

ValueError: could not convert string to float: 'invalid'
```

In [22]:
```
df["Quantity"] = pd.to_numeric(
    df["Quantity"], errors="coerce"
)

df["Price"] = pd.to_numeric(
    df["Price"], errors="coerce"
)

df["Rating"] = pd.to_numeric(
    df["Rating"], errors="coerce"
)
df[["Quantity", "Price", "Rating"]]
```

Out[22]:

	Quantity	Price	Rating
0	2.0	250.0	4.5
1	3.0	180.5	4.0
2	1.0	320.0	5.0
3	NaN	150.0	NaN
4	4.0	NaN	3.5
5	2.0	220.0	NaN
6	NaN	NaN	4.0
7	3.0	275.0	4.5
8	2.0	400.0	NaN
9	5.0	160.0	3.0
10	1.0	300.0	4.5
11	3.0	180.5	4.0

In [23]:
```
print(df[["Quantity", "Price", "Rating"]].dtypes)

display(df[["Quantity", "Price", "Rating"]].isna().sum())
```

```
Quantity    float64
Price       float64
Rating      float64
dtype: object
```

```
Quantity    2
Price       2
Rating      3
dtype: int64
```

In [24]:
```
pd.to_datetime(df["Order_Date"], format="mixed")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[24], line 1
----> 1 pd.to_datetime(df["Order_Date"], format="mixed")

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\tools\datetimes.py:
1239, in to_datetime(arg, errors, dayfirst, yearfirst, utc, format, exact, unit, ori
gin, cache)
   1237         result = arg.map(cache_array).astype(cache_array.dtype)
   1238     else:
-> 1239         values = convert_listlike(arg._values, format)
   1240         result = arg._constructor(values, index=arg.index, name=arg.name)
   1241 elif isinstance(arg, (ABCDataFrame, abc.MutableMapping)):

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\tools\datetimes.py:
486, in _convert_listlike_datetimes(arg, format, name, utc, unit, errors, dayfirst,
yearfirst, exact)
    483 if format is not None and format != "mixed":
    484     return _array_strptime_with_fallback(arg, name, utc, format, exact, erro
rs)
--> 486 result, tz_parsed = objects_to_datetime64(
    487     arg,
    488     dayfirst=dayfirst,
    489     yearfirst=yearfirst,
    490     utc=utc,
    491     errors=errors,
    492 )
    494 if tz_parsed is not None:
    495     # We can take a shortcut since the datetime64 numpy array
    496     # is in UTC
    497     out_unit = np.datetime_data(result.dtype)[0]

File ~\Data-Foundation\Day11\.venv\Lib\site-packages\pandas\core\arrays\datetimes.p
y:2820, in objects_to_datetime64(data, dayfirst, yearfirst, utc, errors, out_unit)
   2817 # if str-dtype, convert
   2818 data = np.asarray(data, dtype=np.object_)
-> 2820 result, tz_parsed = tslib.array_to_datetime(
   2821     data,
   2822     errors=errors,
   2823     utc=utc,
   2824     dayfirst=dayfirst,
   2825     yearfirst=yearfirst,
   2826     creso=abbrev_to_npy_unit(out_unit),
   2827 )
   2829 if tz_parsed is not None:
   2830     # We can take a shortcut since the datetime64 numpy array
   2831     #  is in UTC
   2832     return result, tz_parsed

File pandas/_libs/tslib.pyx:364, in pandas._libs.tslib.array_to_datetime()
--> 364 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/tslib.pyx:600, in pandas._libs.tslib.array_to_datetime()
--> 600 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/tslib.pyx:555, in pandas._libs.tslib.array_to_datetime()
```

```
--> 555 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/tslibs/conversion.pyx:839, in pandas._libs.tslibs.conversion.convert_str_to_tsobject()
--> 839 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/tslibs/parsing.pyx:305, in pandas._libs.tslibs.parsing.parse_datetime_string()
--> 305 'Could not get source, probably due dynamically evaluated source code.'

File pandas/_libs/tslibs/parsing.pyx:213, in pandas._libs.tslibs.parsing._parse_delimited_date()
--> 213 'Could not get source, probably due dynamically evaluated source code.'

File cpython/datetime.pxd:271, in cpython.datetime.datetime_new()
--> 271 'Could not get source, probably due dynamically evaluated source code.'

ValueError: day 31 must be in range 1..28 for month 2 in year 2026
```

Expected: An error occurs because dates such as "31/02/2026" and "not available" cannot be parsed as valid dates.

In [25]:
```
df["Order_Date"] = pd.to_datetime(
    df["Order_Date"],
    format="mixed",
    dayfirst=True,
    errors="coerce"
)

display(df[["Order_ID", "Order_Date"]])
```

	Order_ID	Order_Date
0	101	2026-01-10
1	102	2026-10-02
2	103	2026-03-10
3	104	2026-10-04
4	105	2026-05-10
5	106	NaT
6	107	2026-07-10
7	108	NaT
8	109	2026-10-09
9	110	2026-10-10
10	111	NaT
11	102	2026-10-02

In [26]:
```
print(
    "Invalid or missing dates:",
    df["Order_Date"].isna().sum()
)
```

```
Invalid or missing dates: 3
```

In [27]:
```
clean_df = df.copy()
for col in ["Customer", "City", "Restaurant"]:
    clean_df[col] = clean_df[col].astype("string").str.strip()
clean_df["City"] = clean_df["City"].str.title()
clean_df["Customer"] = clean_df["Customer"].str.title()
clean_df["Restaurant"] = clean_df["Restaurant"].str.title()
duplicates_before = clean_df.duplicated().sum()
clean_df = clean_df.drop_duplicates()
print("Duplicates removed:", duplicates_before)
print("Shape after removing duplicates:", clean_df.shape)
display(clean_df.head())
```

```
Duplicates removed: 1
Shape after removing duplicates: (11, 8)
```

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating
0	101	Arjun	Hyderabad	Spice Hub	2026-01-10	2.0	250.0	4.5
1	102	Priya	Chennai	Food Court	2026-10-02	3.0	180.5	4.0
2	103	Rahul	Hyderabad	Spice Hub	2026-03-10	1.0	320.0	5.0
3	104	Sneha	Bangalore	Green Bowl	2026-10-04	NaN	150.0	NaN
4	105	Kiran	Chennai	Food Court	2026-05-10	4.0	NaN	3.5

In [28]:
```
clean_df["Quantity"] = clean_df["Quantity"].fillna(
    clean_df["Quantity"].median()
)
clean_df["Price"] = clean_df["Price"].fillna(
    clean_df["Price"].mean()
)

clean_df["Rating"] = clean_df["Rating"].fillna(
    clean_df["Rating"].median()
)

clean_df["Quantity"] = clean_df["Quantity"].round().astype(int)

display(clean_df[["Quantity", "Price", "Rating"]])
```

	Quantity	Price	Rating
0	2	250.000000	4.50
1	3	180.500000	4.00
2	1	320.000000	5.00
3	2	150.000000	4.25
4	4	250.611111	3.50
5	2	220.000000	4.25
6	2	250.611111	4.00
7	3	275.000000	4.50
8	2	400.000000	4.25
9	5	160.000000	3.00
10	1	300.000000	4.50

In [29]:
```
clean_df["City"] = clean_df["City"].fillna(
    clean_df["City"].mode().iloc[0]
)

clean_df["Restaurant"] = clean_df["Restaurant"].fillna(
    clean_df["Restaurant"].mode().iloc[0]
)

print("Missing values after imputation:")
display(clean_df.isna().sum())
```

```
Missing values after imputation:
Order_ID      0
Customer      0
City          0
Restaurant    0
Order_Date    3
Quantity      0
Price         0
Rating        0
dtype: int64
```

In [30]:
```
clean_df["City"] = clean_df["City"].fillna(
    clean_df["City"].mode().iloc[0]
)

clean_df["Restaurant"] = clean_df["Restaurant"].fillna(
    clean_df["Restaurant"].mode().iloc[0]
)

print("Missing values after imputation:")
display(clean_df.isna().sum())
```

```
Missing values after imputation:
```

```
Order_ID      0
Customer      0
City          0
Restaurant    0
Order_Date    3
Quantity      0
Price         0
Rating        0
dtype: int64
```

In [31]:
```
clean_df["Year"] = clean_df["Order_Date"].dt.year
clean_df["Month"] = clean_df["Order_Date"].dt.month
clean_df["Month_Name"] = clean_df["Order_Date"].dt.strftime("%B")
clean_df["Day"] = clean_df["Order_Date"].dt.day
clean_df["Quarter"] = clean_df["Order_Date"].dt.quarter

clean_df["Total_Amount"] = (
    clean_df["Quantity"] * clean_df["Price"]
).round(2)

clean_df["Order_Size"] = clean_df["Total_Amount"].apply(
    lambda amount: (
        "High" if amount >= 500
        else "Medium" if amount >= 250
        else "Low"
    )
)

display(clean_df.head(10))
```

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating	
0	101	Arjun	Hyderabad	Spice Hub	2026-01-10	2	250.000000	4.50	2(
1	102	Priya	Chennai	Food Court	2026-10-02	3	180.500000	4.00	2(
2	103	Rahul	Hyderabad	Spice Hub	2026-03-10	1	320.000000	5.00	2(
3	104	Sneha	Bangalore	Green Bowl	2026-10-04	2	150.000000	4.25	2(
4	105	Kiran	Chennai	Food Court	2026-05-10	4	250.611111	3.50	2(
5	106	Divya	Chennai	Spice Hub	NaT	2	220.000000	4.25	
6	107	Aman	Mumbai	Urban Bites	2026-07-10	2	250.611111	4.00	2(
7	108	Neha	Bangalore	Green Bowl	NaT	3	275.000000	4.50	
8	109	Ravi	Mumbai	Urban Bites	2026-10-09	2	400.000000	4.25	2(
9	110	Meena	Chennai	Food Court	2026-10-10	5	160.000000	3.00	2(

A. City-wise analysis

In [35]:
```
city_summary = clean_df.groupby("City").agg(
    Total_Revenue=("Total_Amount", "sum"),
    Total_Orders=("Order_ID", "count"),
    Average_Order_Value=("Total_Amount", "mean"),
    Total_Items=("Quantity", "sum"),
    Average_Rating=("Rating", "mean")
).round(2).sort_values(
    "Total_Revenue", ascending=False
)

display(city_summary)
```

City	Total_Revenue	Total_Orders	Average_Order_Value	Total_Items	Average_Rating
Chennai	2783.94	4	695.98	14	3.69
Mumbai	1301.22	2	650.61	4	4.12
Bangalore	1125.00	2	562.50	5	4.38
Hyderabad	1120.00	3	373.33	4	4.67

B. Restaurant-wise analysis

In [37]:
```
restaurant_summary = clean_df.groupby("Restaurant").agg(
    Total_Revenue=("Total_Amount", "sum"),
    Total_Orders=("Order_ID", "count"),
    Average_Rating=("Rating", "mean"),
    Total_Items=("Quantity", "sum")
).round(2).sort_values(
    "Total_Revenue", ascending=False
)

display(restaurant_summary)
```

Restaurant	Total_Revenue	Total_Orders	Average_Rating	Total_Items
Food Court	2343.94	3	3.50	12
Spice Hub	1560.00	4	4.56	6
Urban Bites	1301.22	2	4.12	4
Green Bowl	1125.00	2	4.38	5

C. Customer-wise analysis

In [38]:
```
customer_summary = clean_df.groupby("Customer").agg(
    Total_Spent=("Total_Amount", "sum"),
    Number_Of_Orders=("Order_ID", "count"),
    Average_Rating=("Rating", "mean")
).round(2).sort_values(
    "Total_Spent", ascending=False
)

display(customer_summary)
```

Customer	Total_Spent	Number_Of_Orders	Average_Rating
Kiran	1002.44	1	3.50
Neha	825.00	1	4.50
Meena	800.00	1	3.00
Ravi	800.00	1	4.25
Priya	541.50	1	4.00
Aman	501.22	1	4.00
Arjun	500.00	1	4.50
Divya	440.00	1	4.25
Rahul	320.00	1	5.00
Sneha	300.00	1	4.25
Vikram	300.00	1	4.50

D. Multiple-column grouping

In [40]:
```
city_restaurant_summary = clean_df.groupby(
    ["City", "Restaurant"]
).agg(
    Total_Revenue=("Total_Amount", "sum"),
    Number_Of_Orders=("Order_ID", "count")
).reset_index()

display(city_restaurant_summary)
```

	City	Restaurant	Total_Revenue	Number_Of_Orders
0	Bangalore	Green Bowl	1125.00	2
1	Chennai	Food Court	2343.94	3
2	Chennai	Spice Hub	440.00	1
3	Hyderabad	Spice Hub	1120.00	3
4	Mumbai	Urban Bites	1301.22	2

E. Identify the highest and lowest performers

In [41]:
```
print(
    "Highest-revenue city:",
    city_summary["Total_Revenue"].idxmax()
)

print(
    "Lowest-revenue city:",
    city_summary["Total_Revenue"].idxmin()
)

print(
    "Top restaurant:",
    restaurant_summary["Total_Revenue"].idxmax()
)

print(
    "Highest-spending customer:",
    customer_summary["Total_Spent"].idxmax()
)

print(
    "Total revenue:",
    round(clean_df["Total_Amount"].sum(), 2)
)
```

```
Highest-revenue city: Chennai
Lowest-revenue city: Hyderabad
Top restaurant: Food Court
Highest-spending customer: Kiran
Total revenue: 6330.16
```

In [42]:
```
delivery_data = pd.DataFrame({
    "City": [
        "Hyderabad",
        "Chennai",
        "Bangalore",
        "Mumbai"
    ],
    "Delivery_Fee": [30, 25, 35, 40]
})
```

```
display(delivery_data)
```

	City	Delivery_Fee
0	Hyderabad	30
1	Chennai	25
2	Bangalore	35
3	Mumbai	40

In [43]:
```
inner_result = pd.merge(
    clean_df,
    delivery_data,
    on="City",
    how="inner"
)

display(
    inner_result[
        ["Order_ID", "City", "Total_Amount", "Delivery_Fee"]
    ]
)
```

	Order_ID	City	Total_Amount	Delivery_Fee
0	101	Hyderabad	500.00	30
1	102	Chennai	541.50	25
2	103	Hyderabad	320.00	30
3	104	Bangalore	300.00	35
4	105	Chennai	1002.44	25
5	106	Chennai	440.00	25
6	107	Mumbai	501.22	40
7	108	Bangalore	825.00	35
8	109	Mumbai	800.00	40
9	110	Chennai	800.00	25
10	111	Hyderabad	300.00	30

In [46]:
```
left_result = pd.merge(
    clean_df,
    delivery_data,
    on="City",
    how="left"
)
```

```
display(left_result.head())
```

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating	
0	101	Arjun	Hyderabad	Spice Hub	2026-01-10	2	250.000000	4.50	2(
1	102	Priya	Chennai	Food Court	2026-10-02	3	180.500000	4.00	2(
2	103	Rahul	Hyderabad	Spice Hub	2026-03-10	1	320.000000	5.00	2(
3	104	Sneha	Bangalore	Green Bowl	2026-10-04	2	150.000000	4.25	2(
4	105	Kiran	Chennai	Food Court	2026-05-10	4	250.611111	3.50	2(

In [47]:
```
left_result["Final_Amount"] = (
    left_result["Total_Amount"] +
    left_result["Delivery_Fee"]
)

display(
    left_result[
        ["Order_ID", "City", "Total_Amount",
         "Delivery_Fee", "Final_Amount"]
    ]
)
```

	Order_ID	City	Total_Amount	Delivery_Fee	Final_Amount
0	101	Hyderabad	500.00	30	530.00
1	102	Chennai	541.50	25	566.50
2	103	Hyderabad	320.00	30	350.00
3	104	Bangalore	300.00	35	335.00
4	105	Chennai	1002.44	25	1027.44
5	106	Chennai	440.00	25	465.00
6	107	Mumbai	501.22	40	541.22
7	108	Bangalore	825.00	35	860.00
8	109	Mumbai	800.00	40	840.00
9	110	Chennai	800.00	25	825.00
10	111	Hyderabad	300.00	30	330.00

In [48]:
```
extra_orders = clean_df.iloc[:2].copy()
extra_orders["Order_ID"] = [201, 202]

combined_df = pd.concat(
    [clean_df, extra_orders],
```

```
    ignore_index=True
)

print("Original cleaned shape:", clean_df.shape)
print("Combined shape:", combined_df.shape)
```

```
Original cleaned shape: (11, 15)
Combined shape: (13, 15)
```

In [50]:
```
display(
    clean_df.sort_values(
        "Total_Amount", ascending=False
    )
)
```

	Order_ID	Customer	City	Restaurant	Order_Date	Quantity	Price	Rating
4	105	Kiran	Chennai	Food Court	2026-05-10	4	250.611111	3.50
7	108	Neha	Bangalore	Green Bowl	NaT	3	275.000000	4.50
8	109	Ravi	Mumbai	Urban Bites	2026-10-09	2	400.000000	4.25
9	110	Meena	Chennai	Food Court	2026-10-10	5	160.000000	3.00
1	102	Priya	Chennai	Food Court	2026-10-02	3	180.500000	4.00
6	107	Aman	Mumbai	Urban Bites	2026-07-10	2	250.611111	4.00
0	101	Arjun	Hyderabad	Spice Hub	2026-01-10	2	250.000000	4.50
5	106	Divya	Chennai	Spice Hub	NaT	2	220.000000	4.25
2	103	Rahul	Hyderabad	Spice Hub	2026-03-10	1	320.000000	5.00
3	104	Sneha	Bangalore	Green Bowl	2026-10-04	2	150.000000	4.25
10	111	Vikram	Hyderabad	Spice Hub	NaT	1	300.000000	4.50

In [51]:
```
display(clean_df["City"].value_counts())
```

```
City
Chennai      4
Hyderabad    3
Bangalore    2
Mumbai       2
Name: count, dtype: Int64
```

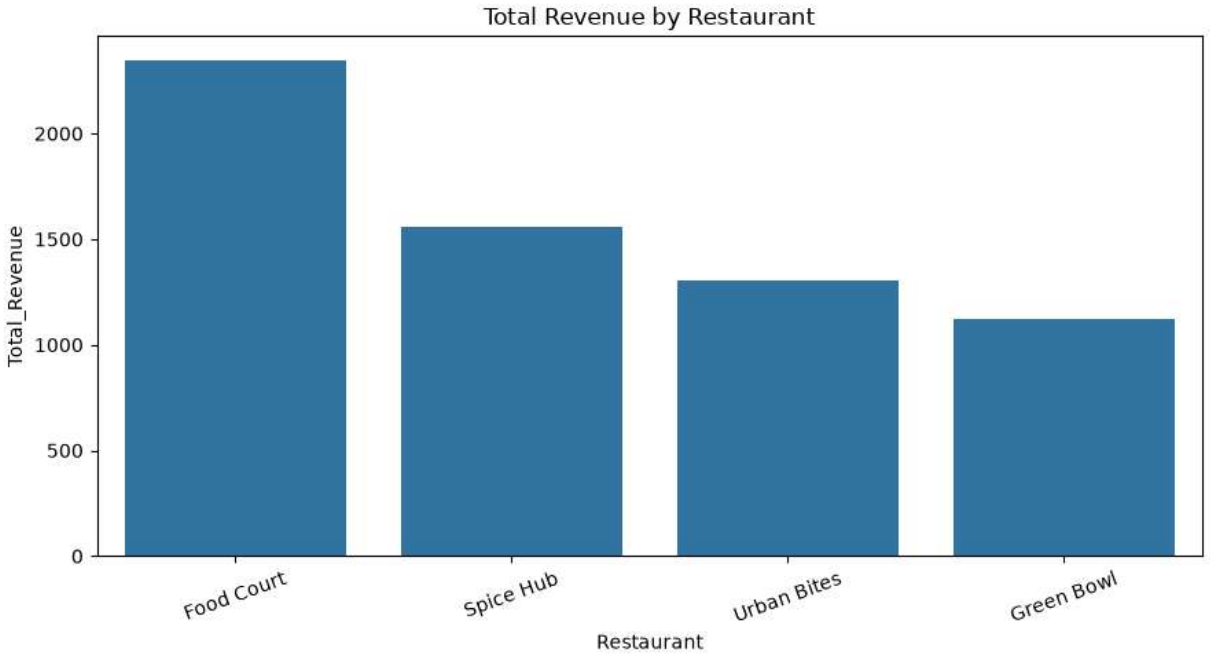
Bar chart

In [52]:
```
plt.figure(figsize=(9, 5))

sns.barplot(
    data=restaurant_summary.reset_index(),
    x="Restaurant",
```

```
    y="Total_Revenue"
)

plt.title("Total Revenue by Restaurant")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

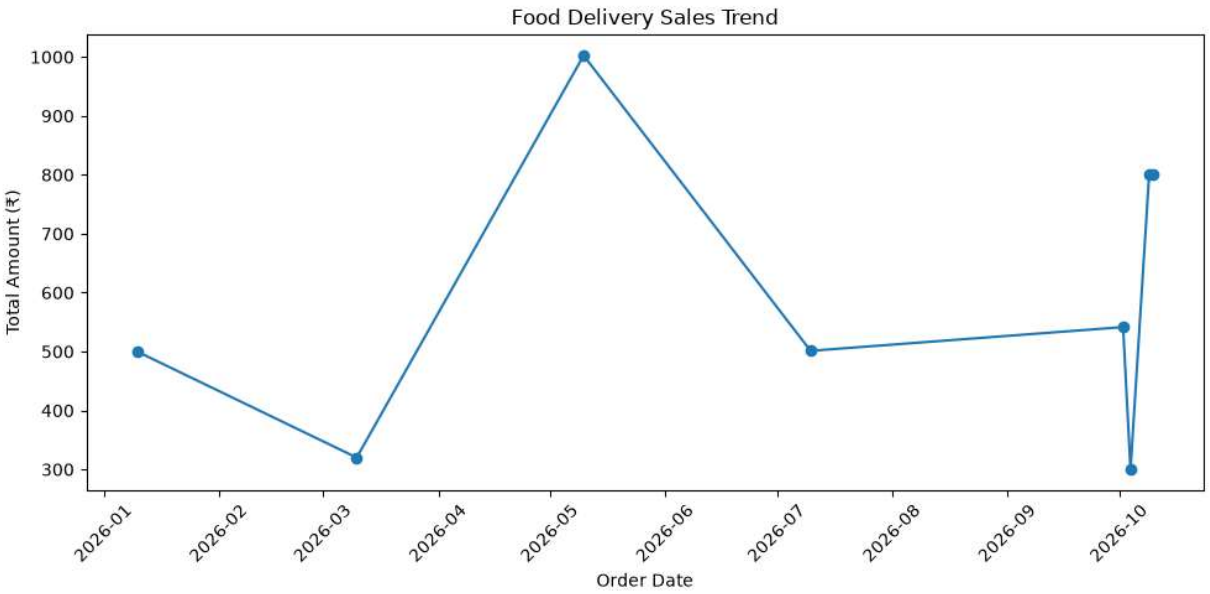


. Line chart — order amounts over time

In [53]:
```
daily_sales = clean_df.groupby("Order_Date")[
    "Total_Amount"
].sum().sort_index()

plt.figure(figsize=(10, 5))
plt.plot(daily_sales.index, daily_sales.values, marker="o")

plt.title("Food Delivery Sales Trend")
plt.xlabel("Order Date")
plt.ylabel("Total Amount (₹)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



C. Histogram — order value distribution

In [54]:
```
plt.figure(figsize=(8, 5))

sns.histplot(
    data=clean_df,
    x="Total_Amount",
    bins=8,
    kde=True
)

plt.title("Distribution of Order Values")
plt.tight_layout()
plt.show()
```

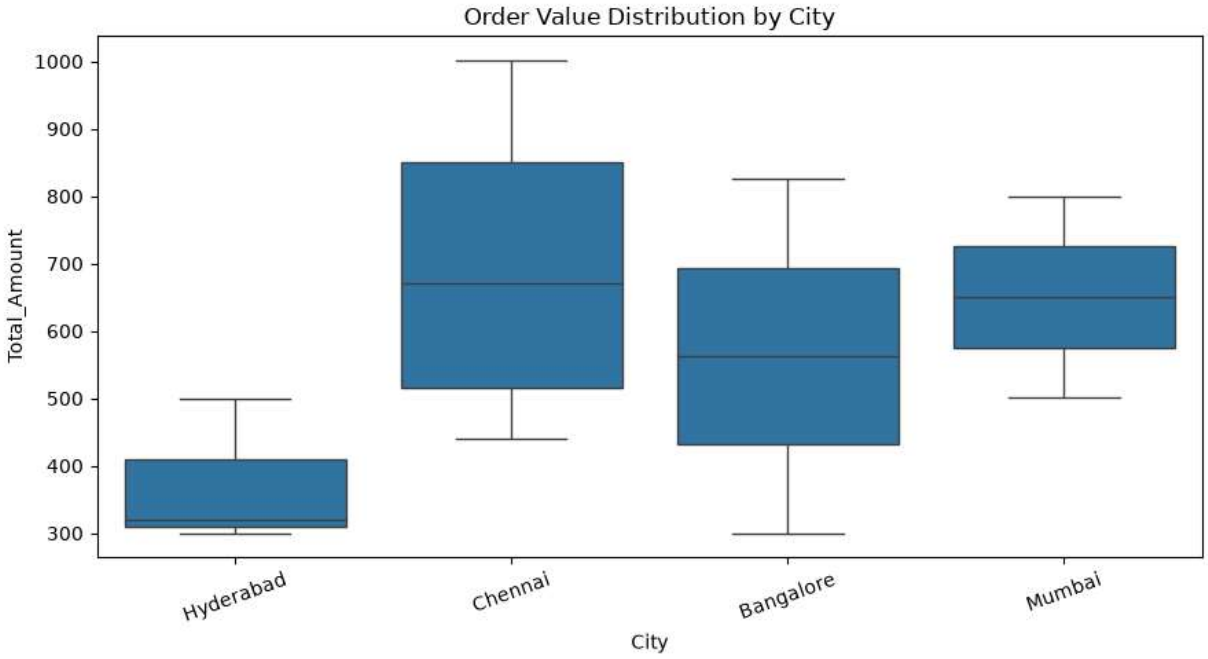


D. Box plot — order value by city

In [55]:
```
plt.figure(figsize=(9, 5))

sns.boxplot(
    data=clean_df,
    x="City",
    y="Total_Amount"
)

plt.title("Order Value Distribution by City")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```



In [56]:
```
clean_df.to_csv(
    "food_delivery_cleaned.csv",
    index=False
)

city_summary.to_csv("city_summary.csv")
restaurant_summary.to_csv("restaurant_summary.csv")
customer_summary.to_csv("customer_summary.csv")

print("All datasets exported successfully!")
```

```
All datasets exported successfully!
```

In [ ]: